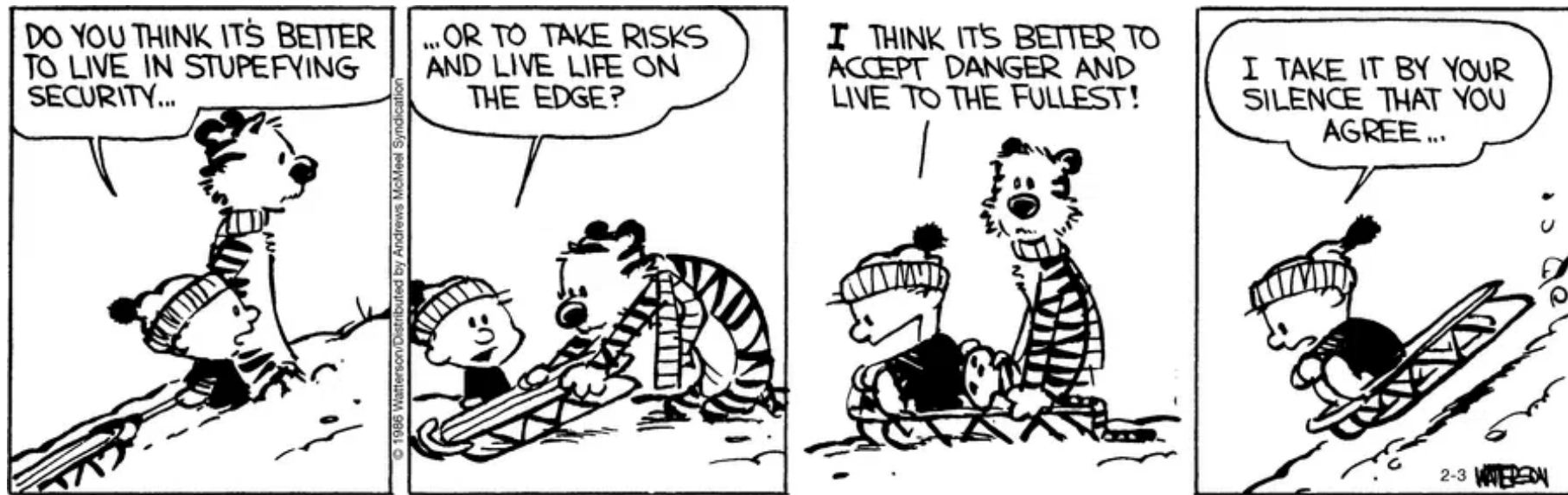


Lecture 14:

Classes + Objects (Part 2)



INFO 1100: Introduction to Programming

Final Project

back to...

Object-Oriented Programming

back to...

Object-Oriented Programming

a programming paradigm that organizes
code around objects

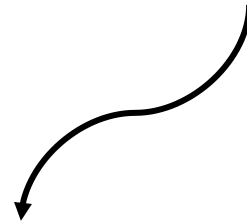
On Thursday...

On Thursday...

objects: units bundling data and behavior

On Thursday...

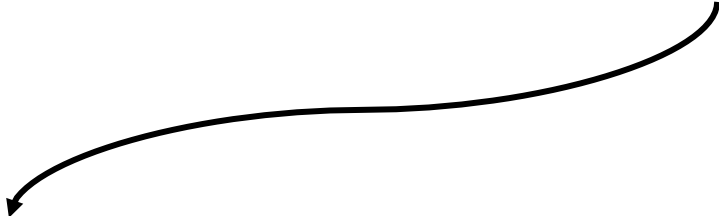
objects: units bundling data and behavior



attributes

On Thursday...

objects: units bundling data and behavior



???

On Thursday...

objects: units bundling data and behavior

methods



Methods

Methods

What are they?

Methods

attributes that are functions

Methods

attributes that are functions

*the **behavior** associated with an object*

how do we write methods?

how do we write methods?

why do we care?

Creating objects

Creating objects

On Thursday...

```
def make_point(x, y, z):  
    p = Point()  
    p.x = x  
    p.y = y  
    p.z = z  
    return p  
  
new_point = make_point(2, 3, 5)
```

Creating objects

but I could still call `Point()`
anywhere in my code!

Creating objects

but I could still call `Point()`
anywhere in my code!

Why could this be
dangerous?

Creating objects

but I could still call `Point()`
anywhere in my code!

leave out an
attribute

Creating objects

but I could still call `Point()`
anywhere in my code!

leave out an
attribute

misspell an
attribute name

Creating objects

but I could still call `Point()`
anywhere in my code!

leave out an
attribute

misspell an
attribute name

store the wrong
type of data

Creating objects

but I could still call `Point()`
anywhere in my code!

can we make it so a function is
called every time we run
`Point()`?

Creating objects

but I could still call `Point()`
anywhere in my code!

method

can we make it so a ~~function~~ is
called every time we run
`Point()`?

`__init__`



`__init__`

the most special method
of them all!

`__init__`

```
class Point:
```

```
    """Represents a 3D point"""
```

__init__

```
class Point:
    """Represents a 3D point"""
    def __init__(self, x, y, z):
        pass
```

`__init__`

```
class Point:
```

```
    """Represents a 3D point"""
```

```
→ def __init__(self, x, y, z):  
    pass
```

indented in the
class definition

`__init__`

```
class Point:
    """Represents a 3D point"""
    def __init__(self, x, y, z):
        pass
```

two underscores

`__init__`

```
class Point:
    """Represents a 3D point"""
    def __init__(self, x, y, z):
        pass
```

two underscores



double underscores

`__init__`

```
class Point:
    """Represents a 3D point"""
    def __init__(self, x, y, z):
        pass
```

two underscores



double underscores



dunders

`__init__`

```
class Point:
    """Represents a 3D point"""
    def __init__(self, x, y, z):
        pass
```

__init__

```
class Point:
    """Represents a 3D point"""
    def __init__(self, x, y, z):
        self.x = x
        self.y = y
        self.z = z
```

`__init__`

```
class Point:
```

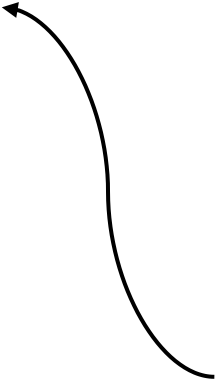
```
    """Represents a 3D point"""
```

```
    def __init__(self, x, y, z):
```

```
        self.x = x
```

```
        self.y = y
```

```
        self.z = z
```



you can add
whatever restrictions
you want here

__init__

```
class Point:
    """Represents a 3D point"""
    def __init__(self, x, y, z):
        self.x = x
        self.y = y
        self.z = z

p = Point(2, 3, 5)
```

`__init__`

```
class Point:
```

```
    """Represents a 3D point"""
```

```
    def __init__(self, x, y, z):
```

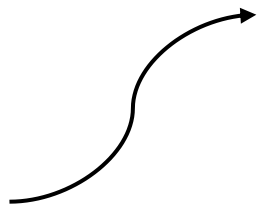
```
        self.x = x
```

```
        self.y = y
```

```
        self.z = z
```

```
p = Point(2, 3, 5)
```

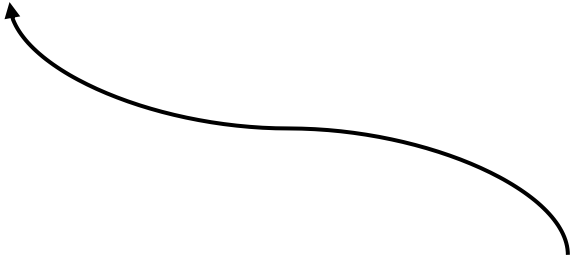
automatically
calls `__init__`!



`__init__`

```
class Point:
    """Represents a 3D point"""
    def __init__(self, x, y, z):
        self.x = x
        self.y = y
        self.z = z
```

what's self?



```
p = Point(2, 3, 5)
```

`__init__`

```
class Point:
```

```
    """Represents a 3D point"""
```

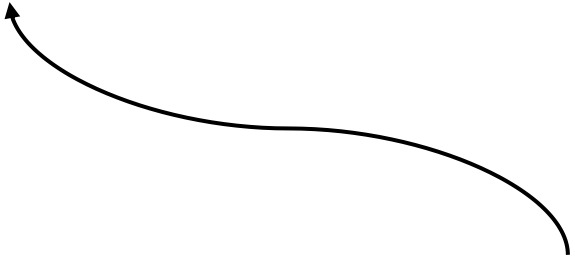
```
    def __init__(self, x, y, z):
```

```
        self.x = x
```

```
        self.y = y
```

```
        self.z = z
```

```
p = Point(2, 3, 5)
```



the object being
created!

`__init__`

```
class Point:
```

```
    """Represents a 3D point"""
```

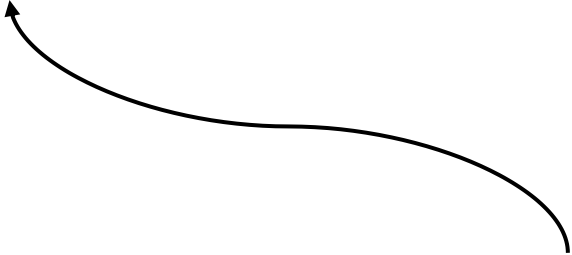
```
    def __init__(self, x, y, z):
```

```
        self.x = x
```

```
        self.y = y
```

```
        self.z = z
```

we use `self` by
convention



```
p = Point(2, 3, 5)
```


__init__

```
class Point:
    """Represents a 3D point"""
    def __init__(self, x, y, z):
        self.x = x
        self.y = y
        self.z = z

p = Point()
```

`__init__`

```
class Point:
```

```
    """Represents a 3D point"""
```

```
    def __init__(self, x, y, z):
```

```
        self.x = x
```

```
        self.y = y
```

```
        self.z = z
```

```
p = Point()
```

leaving out any
parameter would
cause an **Error**



can we only write methods
with special names?

can we only write methods
with special names?

**we can write any
method we want!**

Methods

Methods

```
class Point:
    """Represents a 3D point"""
    def __init__(self, x, y, z):
        ...
```

Methods

```
class Point:
    """Represents a 3D point"""
    def __init__(self, x, y, z):
        ...
```

let's find the distance between any
3D point and the *origin* (0, 0, 0)

Methods

```
class Point:
    """Represents a 3D point"""

    def __init__(self, x, y, z):
        ...

    def distance(self):
        pass
```


Methods

```
class Point:
    """Represents a 3D point"""

    def __init__(self, x, y, z):
        ...

    def distance(self):
        dist = <eq>
```

Methods

```
class Point:
    """Represents a 3D point"""

    def __init__(self, x, y, z):
        ...

    def distance(self):
        dist = (self.x**2+self.y**2
                +self.z**2)**0.5
```

Methods

```
class Point:
    """Represents a 3D point"""

    def __init__(self, x, y, z):
        ...

    def distance(self):
        return (self.x**2+self.y**2
                +self.z**2)**0.5
```

Methods

```
class Point:
    """Represents a 3D point"""

    def __init__(self, x, y, z):
        ...

    def distance(self):
        return (self.x**2+self.y**2
                +self.z**2)**0.5

p = Point(2, 3, 5)
```

Methods

```
class Point:
    """Represents a 3D point"""

    def __init__(self, x, y, z):
        ...

    def distance(self):
        return (self.x**2+self.y**2
                +self.z**2)**0.5

p = Point(2, 3, 5)
dist = p.distance()
```

Methods

```
class Point:
    """Represents a 3D point"""

    def __init__(self, x, y, z):
        ...

    def distance(self):
        return (self.x**2+self.y**2
                +self.z**2)**0.5
```

```
p = Point(2, 3, 5)
dist = p.distance()
```

when we call distance(),
self becomes p

Methods

method call syntax

`<obj>.<method> (<args>)`

Methods

method call syntax

`<obj>.<method> (<args>)`

executes as

`<method> (<obj>, <args>)`

Methods

method call syntax

`<obj>.<method>(<args>)`

executes as

`<method>(<obj>, <args>)`



becomes `self`!

Methods

methods can take multiple
objects as arguments

Methods

```
class Point:  
    """Represents a 3D point"""  
    ...
```

Methods

```
class Point:  
    """Represents a 3D point"""  
    ...
```

let's check if this point is farther
from the origin than another point

Methods

```
class Point:
    """Represents a 3D point"""
    ...

    def is_farther(self, other):
        pass
```

Methods

```
class Point:
    """Represents a 3D point"""
    ...

    def is_further(self, other):
        return self.distance() >
            other.distance()
```

Methods

```
class Point:
    """Represents a 3D point"""
    ...

    def is_further(self, other):
        return self.distance() >
            other.distance()

p1, p2 = Point(2, 3, 5), Point(2, 2, 2)
```

Methods

```
class Point:
    """Represents a 3D point"""
    ...

    def is_farther(self, other):
        return self.distance() >
            other.distance()

p1, p2 = Point(2, 3, 5), Point(2, 2, 2)
p1_vs_p2 = p1.is_farther(p2)
```


Methods

```
class Point:
    """Represents a 3D point"""
    ...

    def is_farther(self, other):
        return self.distance() >
            other.distance()

p1, p2 = Point(2, 3, 5), Point(2, 2, 2)
p1_vs_p2 = p1.is_farther(p2)
```

Methods

```
class Point:
    """Represents a 3D point"""
    ...

    def is_farther(self, other):
        return self.distance() >
            other.distance()

p1, p2 = Point(2, 3, 5), Point(2, 2, 2)
p1_vs_p2 = p1.is_farther(p2)
```

Methods

```
class Point:
    """Represents a 3D point"""
    ...

    def is_farther(self, other):
        return self.distance() >
            other.distance()

p1, p2 = Point(2, 3, 5), Point(2, 2, 2)
p1_vs_p2 = p1.is_farther(p2)
```

**still the same
call syntax!**

Methods

methods that take **self** as a
parameter are called **instance
methods**

Methods

methods that take **self** as a
parameter are called **instance
methods**

they apply to an instance of a class

Methods

methods that take **self** as a parameter are called **instance methods**

they apply to an instance of a class

methods that don't require an instance of a class are called **static**

what if we want to print an object?

what if we want to print an object?

```
p = Point(2, 3, 5)
```


what if we want to print an object?

```
p = Point(2, 3, 5)  
print(p)
```

what if we want to print an object?

```
p = Point(2, 3, 5)
```

```
print(p)
```



```
<Point object at 0x...>
```

what if we want to print an object
readably?

__str__

`__str__`

another special method!

__str__

```
class Point:
    """Represents a 3D point"""
    ...
    def __str__(self):
        pass
```

__str__

```
class Point:
    """Represents a 3D point"""
    ...
    def __str__(self):
        return "(" + str(self.x) + ", " +
                str(self.y) + ", " +
                str(self.z) + ")"
```

__str__

```
class Point:
    """Represents a 3D point"""
    ...
    def __str__(self):
        return "(" + str(self.x) + ", " +
            str(self.y) + ", " +
            str(self.z) + ")"

p = Point(2, 3, 5)
```


__str__

```
class Point:
    """Represents a 3D point"""
    ...
    def __str__(self):
        return "(" + str(self.x) + ", " +
                str(self.y) + ", " +
                str(self.z) + ")"

p = Point(2, 3, 5)
print(p)
```

__str__

```
class Point:
    """Represents a 3D point"""
    ...
    def __str__(self):
        return "(" + str(self.x) + ", " +
            str(self.y) + ", " +
            str(self.z) + ")"

p = Point(2, 3, 5)
print(p)
```

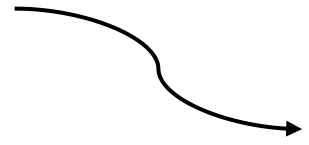
__str__ will automatically be called

__str__

```
class Point:
    """Represents a 3D point"""
    ...
    def __str__(self):
        return "(" + str(self.x) + ", " +
                str(self.y) + ", " +
                str(self.z) + ")"
```

```
p = Point(2, 3, 5)
```

```
print(p)
```



(2, 3, 5)

`__str__`

`__str__` is also automatically
called if you use `str()` with
the object

we may also want to specify what
common operators (e.g. `==`, `+`) mean
for our new objects

we may also want to specify what
common operators (e.g. `==`, `+`) mean
for our new objects

operator overloading

Operator overloading

Operator overloading

for every **operator** there is
some **special method**

Operator overloading

```
class Point:  
    """Represents a 3D point"""  
    ...
```

let's add `==` to our `Point` class

Operator overloading

```
class Point:
    """Represents a 3D point"""
    ...
    def __eq__(self, other):
        pass
```

Operator overloading

```
class Point:
    """Represents a 3D point"""
    ...
    def __eq__(self, other):
        return self.x == other.x and
               self.y == other.y and
               self.z == other.z
```

Operator overloading

```
class Point:
    """Represents a 3D point"""
    ...
    def __eq__(self, other):
        return self.x == other.x and
               self.y == other.y and
               self.z == other.z

p1, p2 = Point(2, 3, 5), Point(2, 2, 2)
```

Operator overloading

```
class Point:
    """Represents a 3D point"""
    ...
    def __eq__(self, other):
        return self.x == other.x and
               self.y == other.y and
               self.z == other.z

p1, p2 = Point(2, 3, 5), Point(2, 2, 2)
print(p1 == p2)
```

Operator overloading

```
class Point:
    """Represents a 3D point"""
    ...
    def __eq__(self, other):
        return self.x == other.x and
               self.y == other.y and
               self.z == other.z

p1, p2 = Point(2, 3, 5), Point(2, 2, 2)
print(p1 == p2)
```

`__eq__` is called

Operator overloading

```
class Point:
    """Represents a 3D point"""
    ...
    def __eq__(self, other):
        return self.x == other.x and
               self.y == other.y and
               self.z == other.z

p1, p2 = Point(2, 3, 5), Point(2, 2, 2)
print(p1 == p2)
```



False

Operator overloading

for every **operator** there is
some **special method**

Operator overloading

for every **operator** there is
some **special method**

`==` $\rightarrow$ `__eq__`

Operator overloading

for every **operator** there is
some **special method**

`==` $\rightarrow$ `__eq__`

`<` $\rightarrow$ `__lt__`

Operator overloading

for every **operator** there is
some **special method**

== → __eq__

< → __lt__

> → __gt__

Operator overloading

for every **operator** there is
some **special method**

== → __eq__

< → __lt__

> → __gt__

+ → __add__

(briefly)

hidden attributes

(briefly)

hidden attributes

we may not want attributes to be accessible
from outside the class

(briefly)

hidden attributes

we may not want attributes to be accessible
from outside the class

*what if the main program
doesn't follow our class's rules?*

(briefly)

hidden attributes

we may not want attributes to be accessible
from outside the class

those attribute names start with _
and have get + set methods

(briefly)

hidden attributes

we may not want attributes to be accessible
from outside the class

those attribute names start with _
and have get + set methods

example in today's lab!

Example 1

```
class Dog:
    """Represents a dog"""

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return self.name + " is " + str(self.age)

d = Dog("Rex", 4)
print(d)
print(d.age * 7)
```

Example 2

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height
    def area(self):
        return self.width * self.height
    def is_square(self):
        return self.width == self.height

r = Rectangle(4, 6)
print(r.area())
print(is_square())
```

Example 3

```
class Weight:
    def __init__(self, grams):
        self.grams = grams
    def __eq__(self, other):
        return self.grams == other.grams
    def __lt__(self, other):
        return self.grams < other.grams

w1, w2, w3 = Weight(50), Weight(75), Weight(75)
print(w1 == w2)
print(w2 == w3)
print(w1 < w2)
```

Questions?

(... I'll wait for 2)

For tomorrow:

(07/22 @ 11:30am)

- Lab 14
- Readings

***Read all four
short sections***